

Отчёт по лабораторной работе №3

Настройка DHCP-сервера

Цыкунова Екатерина Михайловна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	27
4	Контрольные вопросы	28

Список иллюстраций

2.1	Установка DHCP-сервера Kea на виртуальной машине server	6
2.2	Настройка параметров DNS и домена в файле kea-dhcp4.conf	7
2.3	Настройка подсети и диапазона выдаваемых DHCP-адресов	8
2.4	Привязка DHCPv4-сервера Kea к интерфейсу eth1	9
2.5	Добавление PTR-записи DHCP-сервера в обратную DNS-зону	10
2.6	Проверка доступности DHCP-сервера по имени dhcp.emcikunova.net	11
2.7	Настройка firewalld, восстановление контекстов SELinux и запуск Kea DHCPv4	12
2.8	Скрипт настройки маршрутизации виртуальной машины client .	13
2.9	Проверка сетевых интерфейсов клиента после получения DHCP- адреса	14
2.10	Просмотр информации о выданной DHCP-аренде в kea-leases4.csv	15
2.11	Создание TSIG-ключа DHCP_UPDATER для динамического обнов- ления DNS	16
2.12	Разрешение динамического обновления прямой и обратной DNS-зон	17
2.13	TSIG-ключ DHCP_UPDATER в формате конфигурации Kea	18
2.14	Настройка прямых и обратных DDNS-обновлений в kea-dhcp- ddns.conf	19
2.15	Проверка конфигурации и запуск службы Kea DHCP-DDNS	20
2.16	Включение динамического обновления DNS в конфигурации Kea DHCPv4	21
2.17	Перезапуск и проверка состояния Kea DHCPv4 после включения DDNS	22
2.18	Проверка динамически созданной DNS-записи client.emcikunova.net	23
2.19	Копирование конфигурации DHCP и DNS и создание provisioning- скрипта dhcp.sh	25
2.20	Provisioning-скрипт автоматической установки и настройки DHCP-сервера	26

Список таблиц

1 Цель работы

Приобретение практических навыков по установке и конфигурированию DHCP сервера.

2 Выполнение лабораторной работы

На виртуальной машине `server` перехожу в режим суперпользователя и устанавливаю DHCP-сервер Kea командой `dnf -y install kea` (см. рис. Рисунок 2.1).

В процессе установки менеджер пакетов выполняет проверку транзакции и устанавливает основной пакет `kea-3.0.3-1.el10_2.x86_64`, библиотеку `kea-libs`, а также необходимые зависимости, в том числе `libpq`, `log4cplus` и компоненты MariaDB Connector/C. Строки `Transaction check succeeded` и `Transaction test succeeded` показывают успешное прохождение предварительных проверок. Сообщение `Complete!` подтверждает завершение установки без ошибок. После установки в системе становятся доступны службы DHCPv4 и DHCP-DDNS, которые будут использоваться далее.

```
-----
Total                                                                 13 MB/s | 5.2 MB   00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :                                1/1
  Installing     : mariadb-connector-c-config-3.4.4-2.el10_2.noarch 1/6
  Installing     : mariadb-connector-c-3.4.4-2.el10_2.x86_64        2/6
  Installing     : log4cplus-2.1.1-10.el10.x86_64                  3/6
  Installing     : libpq-16.14-1.el10_2.x86_64                     4/6
  Installing     : kea-libs-3.0.3-1.el10_2.x86_64                  5/6
  Running scriptlet: kea-3.0.3-1.el10_2.x86_64                     6/6
  Installing     : kea-3.0.3-1.el10_2.x86_64                     6/6
  Running scriptlet: kea-3.0.3-1.el10_2.x86_64                     6/6

Installed:
  kea-3.0.3-1.el10_2.x86_64          kea-libs-3.0.3-1.el10_2.x86_64
  libpq-16.14-1.el10_2.x86_64      log4cplus-2.1.1-10.el10.x86_64
  mariadb-connector-c-3.4.4-2.el10_2.x86_64  mariadb-connector-c-config-3.4.4-2.el10_2.noarch

Complete!
[root@server.emcikunova.net ~]#
```

Рисунок 2.1: Установка DHCP-сервера Kea на виртуальной машине `server`

Перед изменением настроек сохраняю резервную копию исходного кон-

фигурационного файла командой `cp /etc/kea/kea-dhcp4.conf /etc/kea/kea-dhcp4.conf_$(date -I)`. Затем открываю `/etc/kea/kea-dhcp4.conf` и задаю параметры DNS, которые DHCP-сервер должен передавать клиентам (см. рис. Рисунок 2.2).

В параметре `domain-name-servers` указываю значение `192.168.1.1`. Это адрес виртуальной машины `server`, на которой уже работает настроенный DNS-сервер `BIND`. В результате DHCP-клиенты внутренней сети будут использовать `192.168.1.1` для разрешения доменных имён.

Для DHCP-опции с кодом 15, соответствующей имени домена, задаю значение `emcikunova.net`. Параметр `domain-search` также получает значение `emcikunova.net`. Первая настройка передаёт клиенту основное доменное имя, а вторая задаёт домен поиска. При обращении к узлу по короткому имени система клиента сможет дополнять его суффиксом `emcikunova.net`.



```
144 // "code": 6,
145 // "csv-format": "true",
146 // "space": "dhcp4",
147 // "data": "192.0.2.1, 192.0.2.2"
148 // }
149 // but it's a lot of writing, so it's easier to do this instead:
150 {
151     "name": "domain-name-servers",
152     "data": "192.168.1.1"
153 },
154
155 // Typically people prefer to refer to options by their names, so they
156 // don't need to remember the code names. However, some people like
157 // to use numerical values. For example, option "domain-name" uses
158 // option code 15, so you can reference to it either by
159 // "name": "domain-name" or "code": 15.
160 {
161     "code": 15,
162     "data": "emcikunova.net"
163 },
164
165 // Domain search is also a popular option. It tells the client to
166 // attempt to resolve names within those specified domains. For
167 // example, name "foo" would be attempted to be resolved as
168 // foo.mydomain.example.com and if it fails, then as foo.example.com
169 {
170     "name": "domain-search",
171     "data": "emcikunova.net"
172 },
173
```

Рисунок 2.2: Настройка параметров DNS и домена в файле `kea-dhcp4.conf`

В этом же файле настраиваю IPv4-подсеть, из которой DHCP-сервер будет выдавать адреса клиентам внутренней виртуальной сети (см. рис.

Рисунок 2.3).

В блоке `subnet4` задаю идентификатор подсети `"id": 1` и сеть `"subnet": "192.168.1.0/24"`. Префикс `/24` соответствует маске `255.255.255.0`, поэтому адрес сети равен `192.168.1.0`, а широковещательный адрес — `192.168.1.255`.

В параметре `pools` задаю диапазон `"192.168.1.30 - 192.168.1.199"`. Адреса из этого диапазона Кеа может динамически передавать клиентским узлам. Нижний адрес диапазона `192.168.1.30` впоследствии получает виртуальная машина `client`.

В блоке `option-data` параметр `routers` получает значение `192.168.1.1`. Этот адрес передаётся DHCP-клиенту в качестве шлюза по умолчанию. Поскольку адрес `192.168.1.1` принадлежит внутреннему интерфейсу сервера, клиент сможет направлять через него трафик, предназначенный для других сетей.

```
288 // parameters. One of those parameters is "pools" that is also a list of
289 // structures.
290 "subnet4": [
291     {
292         "id": 1,
293         "subnet": "192.168.1.0/24",
294
295         "pools": [ { "pool": "192.168.1.30 - 192.168.1.199" } ],
296         "option-data": [
297             {
298                 "name": "routers",
299                 "data": "192.168.1.1"
300             }
301         ]
302     }
303 ] // You can add more subnets there.
304 }
305 ],
306
```

Рисунок 2.3: Настройка подсети и диапазона выдаваемых DHCP-адресов

Далее привязываю DHCPv4-сервер к интерфейсу внутренней сети `eth1` (см. рис. Рисунок 2.4). Для этого в секции `interfaces-config` указываю `"interfaces": [ "eth1" ]`.

Ограничение интерфейса необходимо, чтобы DHCP-служба обслуживала именно внутреннюю виртуальную сеть и не принимала DHCP-запросы через интерфейс `eth0`, который используется виртуальной машиной для внешнего сетевого подключения.

После редактирования проверяю синтаксис конфигурации командой `kea-dhcp4 -t /etc/kea/kea-dhcp4.conf`. Затем выполняю `systemctl --system daemon-reload` и разрешаю автоматический запуск сервера командой `systemctl enable kea-dhcp4.service`.

```
27 // and will be ignored by other components.
28 "Dhcp4": {
29     // Add names of your network interfaces to listen on.
30     "interfaces-config": {
31         // See section 8.2.4 for more details. You probably want to add just
32         // interface name (e.g. "eth0" or specific IPv4 address on that
33         // interface name (e.g. "eth0/192.0.2.1").
34         "interfaces": [ "eth1" ]
35     }
36     // Kea DHCPv4 server by default listens using raw sockets. This ensures
37     // all packets, including those sent by directly connected clients
38     // that don't have IPv4 address yet, are received. However, if your
39     // traffic is always relayed, it is often better to use regular
40     // UDP sockets. If you want to do that, uncomment this line:
41     // "dhcp-socket-type": "udp"
42 },
43
44 // Kea supports control channel which is a way to receive management
```

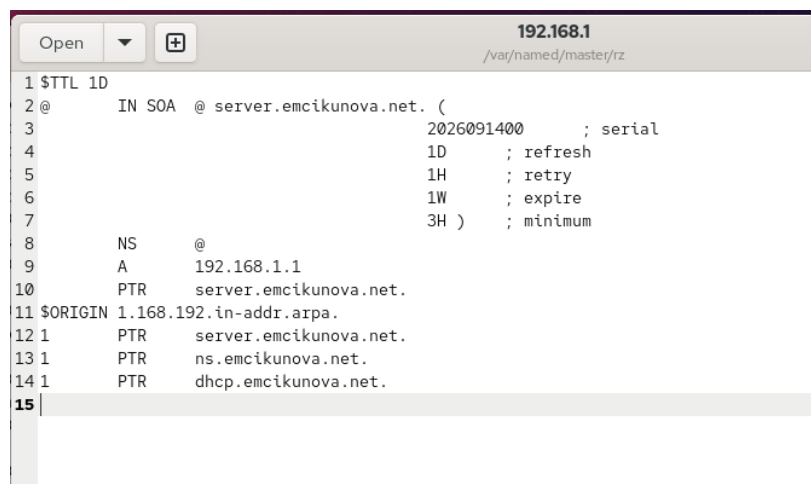
Рисунок 2.4: Привязка DHCPv4-сервера Кеа к интерфейсу eth1

Для обращения к DHCP-серверу по доменному имени добавляю DNS-записи. В файл прямой зоны `/var/named/master/fz/emcikunova.net` вношу запись `dhcp A 192.168.1.1`, связывающую имя `dhcp.emcikunova.net` с IPv4-адресом сервера.

В файл обратной зоны `/var/named/master/rz/192.168.1` добавляю запись `1 PTR dhcp.emcikunova.net`. (см. рис. Рисунок 2.5). Она связывает адрес `192.168.1.1` с доменным именем DHCP-сервера при выполнении обратного DNS-разрешения.

В начале файла установлено `$TTL 1D`, поэтому стандартное время жизни записей зоны составляет один день. В SOA-записи серийный номер изменён на `2026091400`. Первые восемь цифр соответствуют дате `2026-09-14`, а последние две позволяют последовательно увеличивать версию зоны при нескольких изменениях за один день.

В зоне также присутствуют PTR-записи `server.emcikunova.net.`, `ns.emcikunova.net.` и добавленная `dhcp.emcikunova.net.` для адреса с последним октетом `1`.

A screenshot of a text editor window titled '192.168.1' with a subtitle '/var/named/master/rz'. The editor shows a DNS zone file for the reverse lookup zone 192.168.1. The file contains an SOA record, an NS record, an A record for 192.168.1.1, and three PTR records. The third PTR record (line 14) is being added for the DHCP server. The cursor is at the end of line 14.

```
1 $TTL 1D
2 @      IN SOA  @ server.emcikunova.net. (
3                               2026091400      ; serial
4                               1D              ; refresh
5                               1H              ; retry
6                               1W              ; expire
7                               3H )            ; minimum
8      NS      @
9      A        192.168.1.1
10     PTR      server.emcikunova.net.
11 $ORIGIN 1.168.192.in-addr.arpa.
12 1      PTR      server.emcikunova.net.
13 1      PTR      ns.emcikunova.net.
14 1      PTR      dhcp.emcikunova.net.
15
```

Рисунок 2.5: Добавление PTR-записи DHCP-сервера в обратную DNS-зону

После изменения файлов зон перезапускаю DNS-сервер командой `systemctl restart named` и проверяю разрешение нового имени командой `ping dhcp.emcikunova.net` (см. рис. Рисунок 2.6).

Утилита `ping` успешно преобразует имя `dhcp.emcikunova.net` в адрес `192.168.1.1`. В ответах отображается имя `ns.emcikunova.net`, которое также связано с адресом `192.168.1.1` в обратной зоне. Для каждого ICMP-пакета получен ответ с `ttl=64`. Всего передано и принято по 7 пакетов, а значение `0% packet loss` подтверждает отсутствие потерь.

Среднее время прохождения пакета составляет `0.078 ms`, минимальное — `0.015 ms`, максимальное — `0.112 ms`. Успешное разрешение имени и получение ICMP-ответов подтверждают доступность сервера по внутренней сети и корректную обработку созданной DNS-записи.

```
[root@server.emcikunova.net ~]# gedit /var/named/master/fz/emcikunova.net
[root@server.emcikunova.net ~]# gedit /var/named/master/rz/192.168.1
[root@server.emcikunova.net ~]#
[root@server.emcikunova.net ~]# systemctl restart named
[root@server.emcikunova.net ~]# ping dhcp.emcikunova.net
PING dhcp.emcikunova.net (192.168.1.1) 56(84) bytes of data.
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=1 ttl=64 time=0.015 ms
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=2 ttl=64 time=0.056 ms
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=3 ttl=64 time=0.111 ms
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=4 ttl=64 time=0.085 ms
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=5 ttl=64 time=0.084 ms
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=6 ttl=64 time=0.088 ms
64 bytes from ns.emcikunova.net (192.168.1.1): icmp_seq=7 ttl=64 time=0.112 ms
^C
--- dhcp.emcikunova.net ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6199ms
rtt min/avg/max/mdev = 0.015/0.078/0.112/0.031 ms
[root@server.emcikunova.net ~]#
```

Рисунок 2.6: Проверка доступности DHCP-сервера по имени dhcp.emcikunova.net

Далее разрешаю DHCP-трафик в межсетевом экране. Командой `firewall-cmd --add-service=dhcp` добавляю разрешение в текущую конфигурацию `firewalld`, а командой `firewall-cmd --add-service=dhcp --permanent` сохраняю его для последующих запусков системы (см. рис. Рисунок 2.7). В обоих случаях выводится `success`, что подтверждает успешное изменение правил.

После этого восстанавливаю корректные контексты SELinux командами `restorecon -vR /etc`, `restorecon -vR /var/named/` и `restorecon -vR /var/lib/kea/`. В выводе видно, что для файла `/etc/NetworkManager/system-connections/eth1.nmconnection` контекст изменяется с `user_tmp_t` на предназначенный для конфигурации NetworkManager контекст `NetworkManager_etc_rw_t`. Для каталогов `/var/named` и `/var/lib/kea` дополнительных сообщений нет, что означает отсутствие объектов, требующих заметного изменения контекста.

После подготовки меж сетевого экрана и SELinux запускаю DHCPv4-сервер командой `systemctl start kea-dhcp4.service`. Команда завершается без сообщения об ошибке.

```

[root@server.emcikunova.net ~]#
[root@server.emcikunova.net ~]# firewall-cmd --add-service=dhcp
success
[root@server.emcikunova.net ~]# firewall-cmd --add-service=dhcp --permanent
success
[root@server.emcikunova.net ~]# restorecon -vR /etc
Relabeled /etc/NetworkManager/system-connections/eth1.nmconnection from unconfined_u:object_r:user_tmp_t:s0 to unconfined_u:object_r:NetworkManager_etc_rw_t:s0
[root@server.emcikunova.net ~]# restorecon -vR /var/named/
[root@server.emcikunova.net ~]# restorecon -vR /var/lib/kea/
[root@server.emcikunova.net ~]# systemctl start kea-dhcp4.service
[root@server.emcikunova.net ~]#

```

Рисунок 2.7: Настройка firewalld, восстановление контекстов SELinux и запуск Kea DHCPv4

Для подготовки виртуальной машины client создаю скрипт 01-routing.sh, изменяющий её маршрутизацию (см. рис. Рисунок 2.8).

Строка `#!/bin/bash` определяет интерпретатор скрипта. Команда `echo "Provisioning script $0"` выводит имя выполняемого файла и позволяет видеть запуск provisioning-сценария в журнале Vagrant.

Командой `nmcli connection modify "eth1" ipv4.gateway "192.168.1.1"` устанавливаю для соединения eth1 шлюз 192.168.1.1, после чего `nmcli connection up "eth1"` активирует соединение с обновлёнными параметрами.

Для интерфейса eth0 устанавливаю параметры `ipv4.never-default true` и `ipv6.never-default true`. Они запрещают NetworkManager создавать маршрут по умолчанию через eth0 как для IPv4, так и для IPv6. Команды `nmcli connection down eth0` и `nmcli connection up eth0` повторно активируют соединение, чтобы изменения вступили в силу. В результате основной маршрут клиента должен проходить через внутренний интерфейс eth1 и сервер 192.168.1.1.

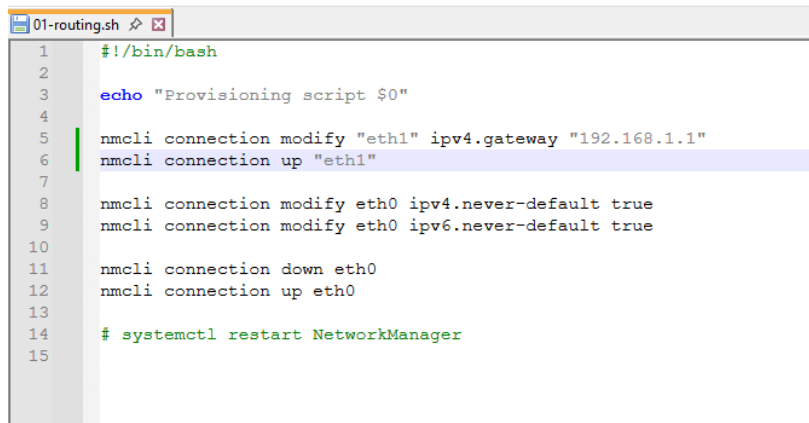
В конфигурацию клиента в Vagrantfile подключаю созданный скрипт:

```

client.vm.provision "client routing",
  type: "shell",
  preserve_order: true,
  run: "always",
  path: "provision/client/01-routing.sh"

```

После сохранения изменений запускаю provisioning клиента.



```
01-routing.sh
1  #!/bin/bash
2
3  echo "Provisioning script $0"
4
5  nmcli connection modify "eth1" ipv4.gateway "192.168.1.1"
6  nmcli connection up "eth1"
7
8  nmcli connection modify eth0 ipv4.never-default true
9  nmcli connection modify eth0 ipv6.never-default true
10
11 nmcli connection down eth0
12 nmcli connection up eth0
13
14 # systemctl restart NetworkManager
15
```

Рисунок 2.8: Скрипт настройки маршрутизации виртуальной машины client

После запуска виртуальной машины client выполняю команду `ifconfig` и анализирую состояние сетевых интерфейсов (см. рис. Рисунок 2.9).

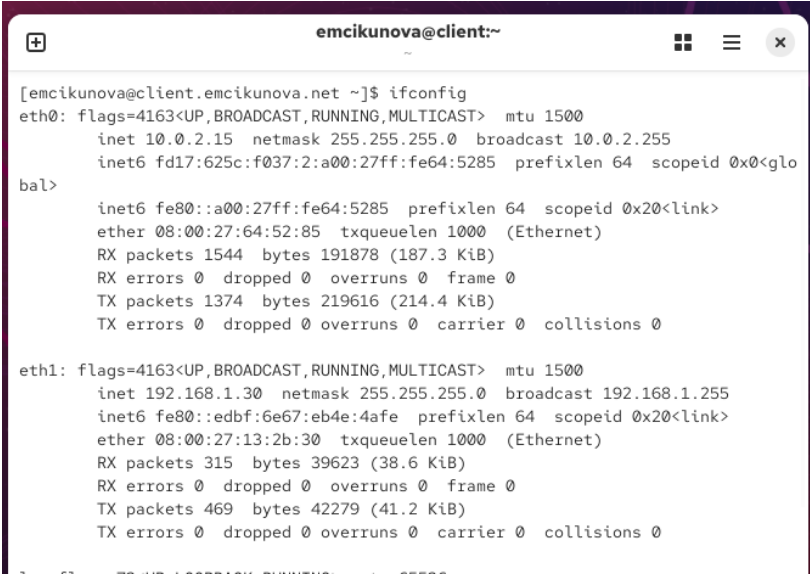
Для `eth0` установлены флаги `UP`, `BROADCAST`, `RUNNING` и `MULTICAST`. Флаг `UP` показывает, что интерфейс включён, `RUNNING` — что он находится в рабочем состоянии, `BROADCAST` обозначает поддержку широковещательной передачи, а `MULTICAST` — групповой передачи. Значение `mtu 1500` задаёт максимальный размер IP-пакета без фрагментации на уровне интерфейса.

Интерфейс `eth0` имеет IPv4-адрес `10.0.2.15`, маску `255.255.255.0` и широковещательный адрес `10.0.2.255`. Этот интерфейс относится к внешней сети виртуальной машины. Строка `ether 08:00:27:64:52:85` содержит его MAC-адрес. В статистике указано `RX packets 1544` и `TX packets 1374`, то есть через интерфейс уже выполнялся обмен данными. Значения `errors 0`, `dropped 0`, `overruns 0` и `collisions 0` показывают отсутствие зарегистрированных ошибок передачи.

Интерфейс `eth1` также находится в состояниях `UP` и `RUNNING`. DHCP-сервер выдал ему IPv4-адрес `192.168.1.30`, маску `255.255.255.0` и широковещательный адрес `192.168.1.255`. Адрес `192.168.1.30` является первым адресом настроенного ранее пула `192.168.1.30 - 192.168.1.199`, поэтому результат соответству-

ет конфигурации Kea.

MAC-адрес eth1 равен 08:00:27:13:2b:30. В статистике интерфейса присутствует RX packets 315 и TX packets 469. Для приёма и передачи также указано по нулю ошибок и отброшенных пакетов. Получение адреса 192.168.1.30 на eth1 подтверждает работоспособность DHCP-сервера во внутренней виртуальной сети.



```
[emcikunova@client.emcikunova.net ~]$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fd17:625c:f037:2:a00:27ff:fe64:5285 prefixlen 64 scopeid 0x0<global>
    ether 08:00:27:64:52:85 txqueuelen 1000 (Ethernet)
    RX packets 1544 bytes 191878 (187.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1374 bytes 219616 (214.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.30 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::edbf:6e67:eb4e:4afe prefixlen 64 scopeid 0x20<link-local>
    ether 08:00:27:13:2b:30 txqueuelen 1000 (Ethernet)
    RX packets 315 bytes 39623 (38.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 469 bytes 42279 (41.2 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Рисунок 2.9: Проверка сетевых интерфейсов клиента после получения DHCP-адреса

На виртуальной машине server просматриваю базу выданных IPv4-аренд Kea командой `cat /var/lib/kea/kea-leases4.csv` (см. рис. Рисунок 2.10).

Первая строка содержит названия полей CSV-файла: `address`, `hwaddr`, `client_id`, `valid_lifetime`, `expire`, `subnet_id`, `fqdn_fwd`, `fqdn_rev`, `hostname`, `state`, `user_context` и `pool_id`.

В записи клиента поле `address` содержит 192.168.1.30 — выданный IPv4-адрес. Поле `hwaddr` равно 08:00:27:13:2b:30 и совпадает с MAC-адресом eth1, отображённым командой `ifconfig`. Это позволяет однозначно сопоставить запись аренды с интерфейсом клиента.

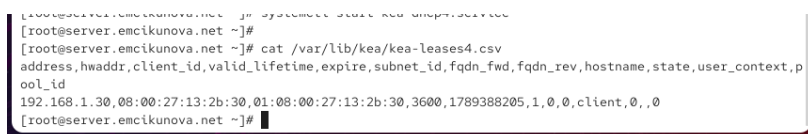
Значение `client_id` содержит идентификатор DHCP-клиента, сформирован-

ный на основе аппаратного адреса. Поле `valid_lifetime` имеет значение 3600, следовательно, срок действия аренды составляет 3600 секунд, то есть один час. Поле `expire` хранит момент окончания аренды в формате Unix time.

Значение `subnet_id` равно 1, что соответствует идентификатору подсети, заданному ранее в `kea-dhcp4.conf`. Поля `fqdn_fwd` и `fqdn_rev` на этом этапе имеют значения 0, поскольку динамическое обновление DNS ещё не настроено. В поле `hostname` записано имя `client`.

Поле `state` имеет значение 0, соответствующее обычному состоянию действующей аренды. `user_context` не заполнено, поскольку дополнительный пользовательский контекст для аренды не задавался. Значение `pool_id` равно 0, так как отдельный идентификатор пула в используемой конфигурации не назначался.

Совпадение адреса и MAC-адреса между `ifconfig` на клиенте и `kea-leases4.csv` на сервере подтверждает, что адрес 192.168.1.30 действительно был выдан сервером Kea.



```
[root@server.emcikunova.net ~]# cat /var/lib/kea/kea-leases4.csv
address,hwaddr,client_id,valid_lifetime,expire,subnet_id,fqdn_fwd,fqdn_rev,hostname,state,user_context,pool_id
192.168.1.30,08:00:27:13:2b:30,01:08:00:27:13:2b:30,3600,1789388205,1,0,0,client,0,,0
[root@server.emcikunova.net ~]#
```

Рисунок 2.10: Просмотр информации о выданной DHCP-аренде в `kea-leases4.csv`

Следующим этапом настраиваю динамическое обновление DNS-зоны при появлении новых DHCP-клиентов. Для безопасной передачи обновлений между Kea и BIND создаю TSIG-ключ (см. рис. Рисунок 2.11).

Командой `mkdir -p /etc/named/keys` создаю каталог для хранения ключей. Затем выполняю `tsig-keygen -a HMAC-SHA512 DHCP_UPDATER > /etc/named/keys/dhcp_updater.key`. Утилита создаёт ключ с именем `DHCP_UPDATER` и алгоритмом `hmac-sha512`, а результат перенаправляется в файл `/etc/named/keys/dhcp_updater.key`.

Командой `cat` проверяю содержимое созданного файла. В нём присутствуют параметры `algorithm hmac-sha512` и `secret`. Значение `secret` представляет сгенерированный секретный ключ, который должен совпадать в настройках BIND и Kea. Сам секрет в тексте отчёта не дублирую.

Командой `chown -R named:named /etc/named/keys/` назначаю владельцем каталога пользователя и группу `named`, после чего открываю `/etc/named.conf` для подключения ключа директивой `include "/etc/named/keys/dhcp_updater.key"`;

```
[root@server.emcikunova.net ~]#  
[root@server.emcikunova.net ~]# mkdir -p /etc/named/keys  
[root@server.emcikunova.net ~]# tsig-keygen -a HMAC-SHA512 DHCP_UPDATER > /etc/named/keys/dhcp_updater.key  
[root@server.emcikunova.net ~]# cat /etc/named/keys/dhcp_updater.key  
key "DHCP_UPDATER" {  
    algorithm hmac-sha512;  
    secret "xl04u02Cu04Np2mymhoideoqzzTJTm5RjS9W2zMGH8TF0uNNhbaa+H+tg0ae0c32OPYJa5M7PrBX16nbJC3Q7A=="  
};  
[root@server.emcikunova.net ~]# chown -R named:named /etc/named/keys/  
[root@server.emcikunova.net ~]# gedit /etc/named.conf  
[root@server.emcikunova.net ~]#
```

Рисунок 2.11: Создание TSIG-ключа DHCP_UPDATER для динамического обновления DNS

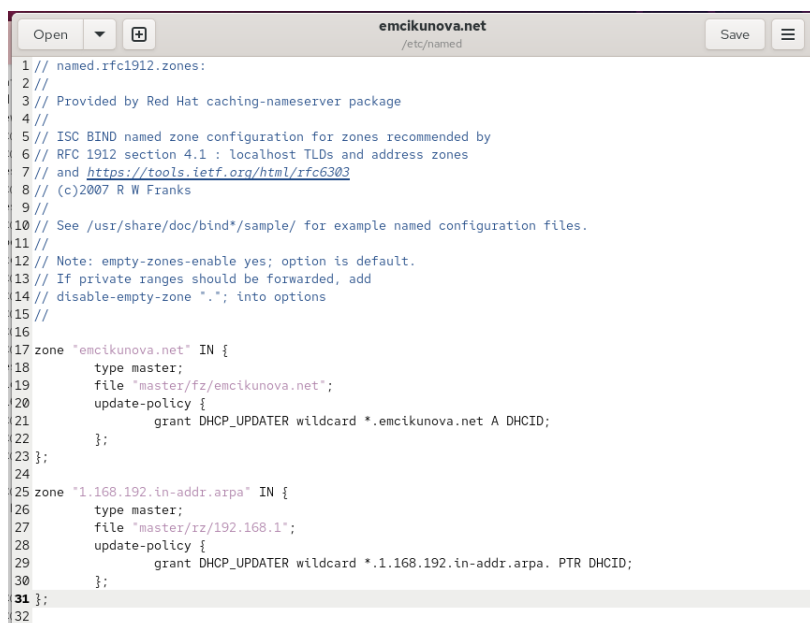
После подключения ключа изменяю описание DNS-зон в файле `/etc/named/emcikunova.net` и разрешаю обновления, подписанные ключом DHCP_UPDATER (см. рис. Рисунок 2.12).

Для прямой зоны `emcikunova.net` сохраняю тип `master` и файл `master/fz/emcikunova.net`. В секции `update-policy` добавляю правило `grant DHCP_UPDATER wildcard *.emcikunova.net A DHCID`; Оно разрешает владельцу ключа DHCP_UPDATER динамически изменять записи типов A и DHCID для имён внутри зоны `emcikunova.net`.

Для обратной зоны `1.168.192.in-addr.arpa` используется файл `master/rz/192.168.1`. Правило `grant DHCP_UPDATER wildcard *.1.168.192.in-addr.arpa. PTR DHCID`; разрешает создание и изменение PTR-записей и связанных DHCID-записей.

Записи A используются для преобразования доменного имени в IPv4-адрес, PTR — для обратного преобразования адреса в имя, а DHCID помогает DHCP/DDNS-механизму связывать динамическую DNS-запись с конкретным DHCP-клиентом и уменьшать вероятность конфликтующих обновлений.

После сохранения файла проверяю конфигурацию BIND командой `named-checkconf` и перезапускаю DNS-сервер командой `systemctl restart named`.



```
1 // named.rfc1912.zones:
2 //
3 // Provided by Red Hat caching-nameserver package
4 //
5 // ISC BIND named zone configuration for zones recommended by
6 // RFC 1912 section 4.1 : localhost TLDs and address zones
7 // and https://tools.ietf.org/html/rfc6303
8 // (c)2007 R W Franks
9 //
10 // See /usr/share/doc/bind*/sample/ for example named configuration files.
11 //
12 // Note: empty-zones-enable yes; option is default.
13 // If private ranges should be forwarded, add
14 // disable-empty-zone "."; into options
15 //
16
17 zone "emcikunova.net" IN {
18     type master;
19     file "master/fz/emcikunova.net";
20     update-policy {
21         grant DHCP_UPDATER wildcard *.emcikunova.net A DHCPID;
22     };
23 };
24
25 zone "1.168.192.in-addr.arpa" IN {
26     type master;
27     file "master/rz/192.168.1";
28     update-policy {
29         grant DHCP_UPDATER wildcard *.1.168.192.in-addr.arpa PTR DHCPID;
30     };
31 };
32
```

Рисунок 2.12: Разрешение динамического обновления прямой и обратной DNS-зон

Для компонента Kea DHCP-DDNS создаю файл `/etc/kea/tsig-keys.json` и помещаю в него тот же ключ `DHCP_UPDATER`, но уже в формате конфигурации Kea (см. рис. Рисунок 2.13).

Массив `tsig-keys` содержит объект с `"name": "DHCP_UPDATER"`, алгоритмом `"hmac-sha512"` и параметром `"secret"`. Секрет должен полностью совпадать со значением в `/etc/named/keys/dhcp_updater.key`, иначе BIND не сможет проверить подпись динамического DNS-обновления.

После сохранения назначаю владельца командой `chown kea:kea /etc/kea/tsig-keys.json` и ограничиваю права доступа командой `chmod 640 /etc/kea/tsig-keys.json`. В результате пользователь `kea` может читать файл, а доступ для остальных пользователей ограничен.



Рисунок 2.13: TSIG-ключ DHCP_UPDATER в формате конфигурации Kea

Далее редактирую `/etc/kea/kea-dhcp-ddns.conf`, настраивая отдельный компонент Kea, отвечающий за динамические DNS-обновления (см. рис. Рисунок 2.14).

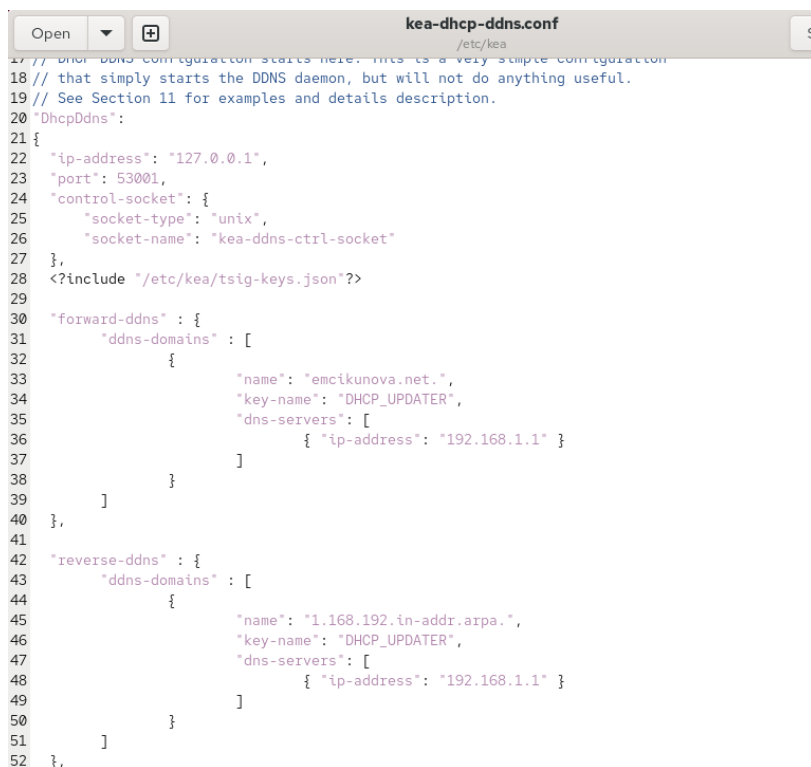
В секции `DhcpDdns` параметр `"ip-address": "127.0.0.1"` указывает, что служба принимает запросы от локальных компонентов через `loopback`-интерфейс. Порт `"53001"` используется для взаимодействия между DHCPv4 и DHCP-DDNS.

В `control-socket` указан Unix-сокет `/run/kea/kea-ddns-ctrl-socket`, предназначенный для локального управления службой. Директива `<?include "/etc/kea/tsig-keys.json"?>` подключает созданный ранее TSIG-ключ.

В секции `forward-ddns` указываю прямую зону `"emcikunova.net."`, ключ `"DHCP_UPDATER"` и DNS-сервер с адресом `192.168.1.1`. Эта секция отвечает за создание записей вида `имя → IPv4-адрес`.

В секции `reverse-ddns` задаю обратную зону `"1.168.192.in-addr.arpa."`, тот же TSIG-ключ и сервер `192.168.1.1`. Она предназначена для создания PTR-записей вида `IPv4-адрес → имя`.

В именах обеих DNS-зон присутствует завершающая точка. Она указывает на полное доменное имя зоны и необходима для корректного сопоставления обновления с зоной BIND.



```

17 // DHCP DDNS configuration starts here. This is a very simple configuration
18 // that simply starts the DDNS daemon, but will not do anything useful.
19 // See Section 11 for examples and details description.
20 "DhcpDdns":
21 {
22   "ip-address": "127.0.0.1",
23   "port": 53001,
24   "control-socket": {
25     "socket-type": "unix",
26     "socket-name": "kea-ddns-ctrl-socket"
27   },
28   <?include "/etc/kea/tsig-keys.json"?>
29
30   "forward-ddns" : {
31     "ddns-domains" : [
32       {
33         "name": "emcikunova.net.",
34         "key-name": "DHCP_UPDATER",
35         "dns-servers": [
36           { "ip-address": "192.168.1.1" }
37         ]
38       }
39     ]
40   },
41
42   "reverse-ddns" : {
43     "ddns-domains" : [
44       {
45         "name": "1.168.192.in-addr.arpa.",
46         "key-name": "DHCP_UPDATER",
47         "dns-servers": [
48           { "ip-address": "192.168.1.1" }
49         ]
50       }
51     ]
52   },

```

Рисунок 2.14: Настройка прямых и обратных DDNS-обновлений в kea-dhcp-ddns.conf

После редактирования назначаю владельца файла командой `chown kea:kea /etc/kea/kea-dhcp-ddns.conf` и проверяю конфигурацию командой `kea-dhcp-ddns -t /etc/kea/kea-dhcp-ddns.conf` (см. рис. Рисунок 2.15).

В диагностическом выводе присутствует сообщение `Configuration check successful`, следовательно, синтаксических ошибок в конфигурации не обнаружено. Также указано, что служба прослушивает адрес `127.0.0.1`, порт `53001` и использует UDP, что соответствует заданным параметрам.

Командой `systemctl enable --now kea-dhcp-ddns.service` одновременно разрешаю автоматический запуск службы при загрузке системы и запускаю её в текущем сеансе. Создание символической ссылки в `multi-user.target.wants` подтверждает включение автозапуска.

Команда `systemctl status kea-dhcp-ddns.service` показывает состояние `active (running)`. Поле `Loaded` содержит `enabled`, а строка `Status: "Dispatching`

packets..." указывает, что процесс запущен и готов обрабатывать сообщения. В журнале присутствуют сообщения DHCP_DDNS_STARTED и Started kea-dhcp-ddns.service, подтверждающие штатный запуск компонента.

```
[root@server.emcikunova.net ~]#
[root@server.emcikunova.net ~]# chown kea:kea /etc/kea/kea-dhcp-ddns.conf
[root@server.emcikunova.net ~]# kea-dhcp-ddns -t /etc/kea/kea-dhcp-ddns.conf
2026-09-14 11:39:28.704 INFO [kea-dhcp-ddns.dctl/17924.140627666033984] DCTL_CONFIG_CHECK_COMPLETE serv
er has completed configuration check: listening on 127.0.0.1, port 53001, using UDP, result: success(0),
text=Configuration check successful
[root@server.emcikunova.net ~]# systemctl enable --now kea-dhcp-ddns.service
Created symlink '/etc/systemd/system/multi-user.target.wants/kea-dhcp-ddns.service' -> '/usr/lib/systemd/
system/kea-dhcp-ddns.service'.
[root@server.emcikunova.net ~]# systemctl status kea-dhcp-ddns.service
● kea-dhcp-ddns.service - Kea DHCP-DDNS Server
   Loaded: loaded (/usr/lib/systemd/system/kea-dhcp-ddns.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-14 11:39:57 UTC; 11s ago
 Invocation: 19cdbbf8497d4732a168cdb8b03e1e98
    Docs: man:kea-dhcp-ddns(8)
   Main PID: 18177 (kea-dhcp-ddns)
    Status: "Dispatching packets..."
     Tasks: 5 (limit: 10233)
    Memory: 2.3M (peak: 7M)
       CPU: 12ms
   CGroup: /system.slice/kea-dhcp-ddns.service
           └─18177 /usr/sbin/kea-dhcp-ddns -c /etc/kea/kea-dhcp-ddns.conf

Sep 14 11:39:57 server.emcikunova.net systemd[1]: Starting kea-dhcp-ddns.service - Kea DHCP-DDNS Server.
Sep 14 11:39:57 server.emcikunova.net kea-dhcp-ddns[18177]: 2026-09-14 11:39:57.262 INFO [kea-dhcp-ddns
Sep 14 11:39:57 server.emcikunova.net kea-dhcp-ddns[18177]: INFO COMMAND_ACCEPTOR_START Starting to ac
Sep 14 11:39:57 server.emcikunova.net kea-dhcp-ddns[18177]: INFO DCTL_CONFIG_COMPLETE server has compl
Sep 14 11:39:57 server.emcikunova.net kea-dhcp-ddns[18177]: INFO DHCP_DDNS_STARTED Kea DHCP-DDNS serv
Sep 14 11:39:57 server.emcikunova.net systemd[1]: Started kea-dhcp-ddns.service - Kea DHCP-DDNS Server.
[root@server.emcikunova.net ~]#
```

Рисунок 2.15: Проверка конфигурации и запуск службы Kea DHCP-DDNS

После запуска DHCP-DDNS возвращаюсь к /etc/kea/kea-dhcp4.conf и разре-
шаю DHCPv4-серверу отправлять запросы на динамическое изменение DNS
(см. рис. Рисунок 2.16).

В секции "dhcp-ddns" устанавливаю "enable-updates": true, включая ме-
ханизм DDNS. Параметр "ddns-qualifying-suffix": "emcikunova.net" задаёт
доменный суффикс, который добавляется к имени DHCP-клиента. Например,
имя client преобразуется в полное имя client.emcikunova.net.

Параметр "ddns-override-client-update": true разрешает серверу самостоя-
тельно управлять обновлением DNS-записей, даже если клиент передаёт соб-
ственные параметры обновления FQDN. В результате информация о получен-
ной DHCP-аренде может автоматически передаваться компоненту kea-dhcp-
ddns, который затем формирует подписанный TSIG-запрос к BIND.

После сохранения проверяю конфигурацию командой kea-dhcp4 -t
/etc/kea/kea-dhcp4.conf.



```

32 // interface name (e.g. "eth0" or specific IPv4 address on that
33 // interface name (e.g. "eth0/192.0.2.1").
34 "interfaces": [ "eth1" ]
35
36 // Kea DHCPv4 server by default listens using raw sockets. This ensures
37 // all packets, including those sent by directly connected clients
38 // that don't have IPv4 address yet, are received. However, if your
39 // traffic is always relayed, it is often better to use regular
40 // UDP sockets. If you want to do that, uncomment this line:
41 // "dhcp-socket-type": "udp"
42 },
43
44 "dhcp-ddns": {
45     "enable-updates": true
46 },
47 "ddns-qualifying-suffix": "emcikunova.net",
48 "ddns-override-client-update": true,
49
50 // Kea supports control channel, which is a way to receive management

```

Рисунок 2.16: Включение динамического обновления DNS в конфигурации Kea DHCPv4

Для применения новой конфигурации выполняю `systemctl restart kea-dhcp4.service`, после чего проверяю состояние командой `systemctl status kea-dhcp4.service` (см. рис. Рисунок 2.17).

Строка `Loaded` показывает, что служба загружена и имеет состояние `enabled`, то есть включена для автоматического запуска. Значение `Active: active (running)` подтверждает работу DHCPv4-сервера.

Основной процесс имеет PID 18695. Строка `Status: "Dispatching packets..."` означает, что Kea перешёл в рабочее состояние и ожидает DHCP-пакеты. В нижней части вывода присутствуют сообщения о запуске `kea-dhcp4.service` без сообщений об ошибках.

```

[root@server.emcikunova.net ~]#
[root@server.emcikunova.net ~]# systemctl restart kea-dhcp4.service
[root@server.emcikunova.net ~]# systemctl status kea-dhcp4.service
● kea-dhcp4.service - Kea DHCPv4 Server
   Loaded: loaded (/usr/lib/systemd/system/kea-dhcp4.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-14 11:42:58 UTC; 11s ago
  Invocation: 50e59e8cb68c4b0d9b1e5b595e42e7e2
     Docs: man:kea-dhcp4(8)
    Main PID: 18695 (kea-dhcp4)
      Status: "Dispatching packets..."
        Tasks: 7 (limit: 10233)
       Memory: 2.5M (peak: 7M)
          CPU: 13ms
    CGroup: /system.slice/kea-dhcp4.service
            └─18695 /usr/sbin/kea-dhcp4 -c /etc/kea/kea-dhcp4.conf

Sep 14 11:42:58 server.emcikunova.net systemd[1]: Starting kea-dhcp4.service - Kea DHCPv4 Server...
Sep 14 11:42:58 server.emcikunova.net kea-dhcp4[18695]: 2026-09-14 11:42:58.584 INFO [kea-dhcp4.dhcp4/8
Sep 14 11:42:58 server.emcikunova.net kea-dhcp4[18695]: 2026-09-14 11:42:58.585 INFO [kea-dhcp4.command
Sep 14 11:42:58 server.emcikunova.net systemd[1]: Started kea-dhcp4.service - Kea DHCPv4 Server.
[root@server.emcikunova.net ~]#

```

Рисунок 2.17: Перезапуск и проверка состояния Kea DHCPv4 после включения DDNS

На виртуальной машине client принудительно переполучаю сетевые параметры. Для этого выполняю `nmcli connection down eth1`, а затем `nmcli connection up eth1` (см. рис. Рисунок 2.18). NetworkManager сообщает `successfully deactivated` и `successfully activated`, подтверждая успешное переподключение интерфейса.

После нового получения DHCP-аренды проверяю появление динамической DNS-записи командой `dig @192.168.1.1 client.emcikunova.net`.

В первой строке вывода `DiG 9.18.33` указана версия утилиты, адрес `@192.168.1.1` задаёт DNS-сервер, к которому направляется запрос, а `client.emcikunova.net` является проверяемым именем.

В секции HEADER значение `opcode: QUERY` обозначает обычный DNS-запрос, а `status: NOERROR` подтверждает его успешную обработку. Идентификатор запроса равен 29917.

Флаги `qr aa rd ra` показывают, что получено сообщение-ответ (`qr`), ответ является авторитетным для запрашиваемой зоны (`aa`), клиент запросил рекурсию (`rd`), а сервер поддерживает рекурсивную обработку (`ra`). Значения `QUERY: 1` и `ANSWER: 1` указывают соответственно на один запрос и одну запись в ответе. Секции `AUTHORITY` нет, а в `ADDITIONAL` присутствует одна дополнительная запись.

В OPT PSEUDOSECTION указано использование EDNS версии 0 и максимальный размер UDP-сообщения 1232 байта. Поле COOKIE используется как дополнительный механизм сопоставления DNS-запроса и ответа.

В QUESTION SECTION запрашивается запись типа A для client.emcikunova.net.

В ANSWER SECTION содержится строка client.emcikunova.net. 1200 IN A 192.168.1.30. Значение 192.168.1.30 совпадает с адресом, ранее выданным клиенту DHCP-сервером. TTL записи равен 1200 секунд.

Строка Query time: 0 msec показывает практически мгновенную обработку локального запроса. SERVER: 192.168.1.1#53(192.168.1.1) (UDP) подтверждает, что ответ получен от настроенного DNS-сервера на стандартном порту 53 по UDP.

Наличие авторитетного ответа с записью client.emcikunova.net → 192.168.1.30 подтверждает, что связка Kea DHCPv4, Kea DHCP-DDNS и BIND успешно выполняет автоматическое обновление прямой DNS-зоны после получения клиентом DHCP-адреса.

```
[emcikunova@client.emcikunova.net ~]$  
[emcikunova@client.emcikunova.net ~]$ nmcli connection down eth1  
Connection 'eth1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/5)  
[emcikunova@client.emcikunova.net ~]$ nmcli connection up eth1  
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/7)  
[emcikunova@client.emcikunova.net ~]$ dig @192.168.1.1 client.emcikunova.net  
  
; <<>> DiG 9.18.33 <<>> @192.168.1.1 client.emcikunova.net  
; (1 server found)  
;; global options: +cmd  
;; Got answer:  
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29917  
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags:; udp: 1232  
; COOKIE: 60cbf970d263840f010000006aa7de1c932c4f5dd3cc2792 (good)  
;; QUESTION SECTION:  
;client.emcikunova.net. IN A  
  
;; ANSWER SECTION:  
client.emcikunova.net. 1200 IN A 192.168.1.30  
  
;; Query time: 0 msec  
;; SERVER: 192.168.1.1#53(192.168.1.1) (UDP)  
;; WHEN: Mon Sep 14 11:44:27 UTC 2026  
;; MSG SIZE rcvd: 94  
  
[emcikunova@client.emcikunova.net ~]$
```

Рисунок 2.18: Проверка динамически созданной DNS-записи client.emcikunova.net

Для фиксации выполненной настройки в provisioning-конфигурации

Vagrant перехожу на сервере в каталог `/vagrant/provision/server/` и создаю структуру каталогов для DHCP-файлов (см. рис. Рисунок 2.19).

Командой `mkdir -p /vagrant/provision/server/dhcp/etc/kea` создаю каталог назначения, после чего `cp -R /etc/kea/* /vagrant/provision/server/dhcp/etc/kea/` копирует конфигурационные файлы Kea.

Затем перехожу в `/vagrant/provision/server/dns/` и обновляю сохранённую конфигурацию DNS. Командой `cp -R /var/named/* /vagrant/provision/server/dns/var/named/` копирую содержимое `/var/named`, включая изменённые файлы зон. При запросе на перезапись файлов прямой и обратной зон отвечаю `y`.

При копировании появляются сообщения `cannot create symbolic link ... Protocol error` для `named.ca`, `named.empty`, `named.localhost` и `named.loopback`. Ошибка относится к созданию символических ссылок внутри разделяемого каталога `/vagrant` и связана с ограничениями используемой общей файловой системы виртуальной машины. При этом обычные конфигурационные файлы и изменённые файлы зон копируются отдельно.

Командой `cp -R /etc/named/* /vagrant/provision/server/dns/etc/named/` копирую конфигурацию из `/etc/named`, включая файл описания зоны `emcikunova.net`.

После этого возвращаюсь в `/vagrant/provision/server/`, создаю файл `dhcp.sh` командой `touch dhcp.sh` и назначаю ему право на выполнение командой `chmod +x dhcp.sh`.

```

[root@server.emcikunova.net ~]# cd /vagrant/provision/server/
[root@server.emcikunova.net server]# mkdir -p /vagrant/provision/server/dhcp/etc/kea
[root@server.emcikunova.net server]# cp -R /etc/kea/* /vagrant/provision/server/dhcp/etc/kea/
[root@server.emcikunova.net server]# cd /vagrant/provision/server/dns/
[root@server.emcikunova.net dns]# cp -R /var/named/* /vagrant/provision/server/dns/var/named/
cp: overwrite '/vagrant/provision/server/dns/var/named/master/fz/emcikunova.net'? y
cp: overwrite '/vagrant/provision/server/dns/var/named/master/rz/192.168.1'? y
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.ca': Protocol error
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.empty': Protocol error
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.localhost': Protocol error
cp: cannot create symbolic link '/vagrant/provision/server/dns/var/named/named.loopback': Protocol error
[root@server.emcikunova.net dns]# cp -R /etc/named/* /vagrant/provision/server/dns/etc/named/
cp: overwrite '/vagrant/provision/server/dns/etc/named/emcikunova.net'? y
[root@server.emcikunova.net dns]# cd /vagrant/provision/server/
[root@server.emcikunova.net server]# touch dhcp.sh
[root@server.emcikunova.net server]# chmod +x dhcp.sh
[root@server.emcikunova.net server]#

```

Рисунок 2.19: Копирование конфигурации DHCP и DNS и создание provisioning-скрипта dhcp.sh

В созданном файле `dhcp.sh` фиксирую основные действия, необходимые для автоматического развёртывания DHCP-сервера (см. рис. Рисунок 2.20).

Строка `#!/bin/bash` определяет Bash как интерпретатор. Команда `echo "Provisioning script $0"` выводит информацию о запуске скрипта. Далее `dnf -y install kea` обеспечивает автоматическую установку Kea без интерактивного подтверждения.

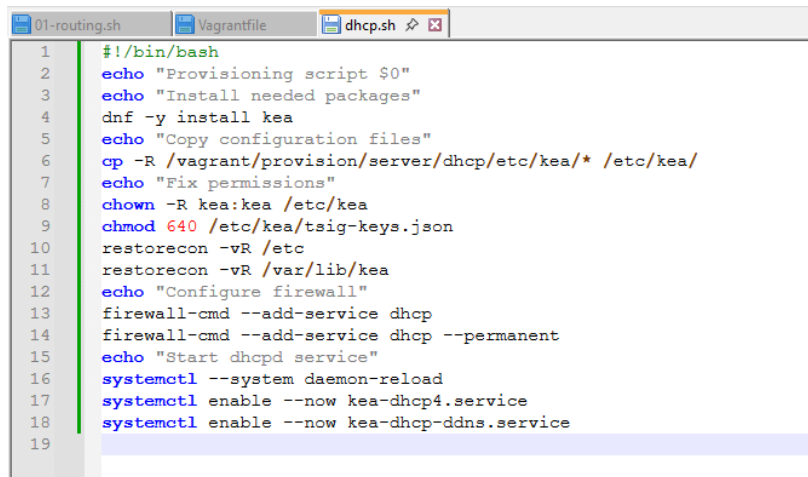
Команда `cp -R /vagrant/provision/server/dhcp/etc/kea/* /etc/kea/` восстанавливает подготовленные конфигурационные файлы Kea. После копирования команда `chown -R kea:kea /etc/kea` назначает корректного владельца, а `chmod 640 /etc/kea/tsig-keys.json` ограничивает доступ к файлу с TSIG-секретом.

Команды `restorecon -vR /etc` и `restorecon -vR /var/lib/kea` восстанавливают контексты SELinux для конфигурационных и рабочих файлов DHCP-сервера.

Далее скрипт выполняет `firewall-cmd --add-service dhcp` и `firewall-cmd --add-service dhcp --permanent`, разрешая DHCP-трафик как в текущей конфигурации `firewalld`, так и после перезагрузки системы.

Команда `systemctl --system daemon-reload` перечитывает конфигурацию `systemd`. Затем `systemctl enable --now kea-dhcp4.service` и `systemctl enable --now kea-dhcp-ddns.service` одновременно включают автозапуск и запускают

DHCPv4- и DHCP-DDNS-службы.



```
01-routing.sh Vagrantfile dhcp.sh
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Install needed packages"
4  dnf -y install kea
5  echo "Copy configuration files"
6  cp -R /vagrant/provision/server/dhcp/etc/kea/* /etc/kea/
7  echo "Fix permissions"
8  chown -R kea:kea /etc/kea
9  chmod 640 /etc/kea/tsig-keys.json
10 restorecon -vR /etc
11 restorecon -vR /var/lib/kea
12 echo "Configure firewall"
13 firewall-cmd --add-service dhcp
14 firewall-cmd --add-service dhcp --permanent
15 echo "Start dhcpd service"
16 systemctl --system daemon-reload
17 systemctl enable --now kea-dhcp4.service
18 systemctl enable --now kea-dhcp-ddns.service
19
```

Рисунок 2.20: Provisioning-скрипт автоматической установки и настройки DHCP-сервера

Для запуска созданного provisioning-скрипта при развёртывании виртуальной машины server добавляю в раздел её конфигурации в Vagrantfile:

```
server.vm.provision "server dhcp",
  type: "shell",
  preserve_order: true,
  path: "provision/server/dhcp.sh"
```

Параметр type: "shell" указывает Vagrant на выполнение shell-скрипта, preserve_order: true сохраняет заданную последовательность provisioning-сценариев, а path: "provision/server/dhcp.sh" задаёт путь к подготовленному файлу.

В результате конфигурация проекта содержит необходимые файлы Kea и BIND, параметры внутренней DHCP-сети, механизм динамического обновления DNS с аутентификацией TSIG и provisioning-скрипт для автоматического восстановления настроек. Проверка на клиентской машине показала получение адреса 192.168.1.30 из настроенного диапазона, а запрос dig подтвердил автоматическое появление записи client.emcikunova.net в прямой DNS-зоне.

3 Вывод

В результате выполнения работы на виртуальной машине `server` установлен и настроен DHCP-сервер Kea. Для внутренней сети `192.168.1.0/24` задан диапазон динамически выдаваемых адресов `192.168.1.30–192.168.1.199`, в качестве шлюза и DNS-сервера указан адрес `192.168.1.1`. Работа DHCP-сервера ограничена внутренним интерфейсом `eth1`, для службы настроен межсетевой экран и включён автоматический запуск.

На виртуальной машине `client` выполнена настройка маршрутизации через внутренний интерфейс `eth1`. После подключения клиента DHCP-сервер успешно выдал ему адрес `192.168.1.30`, что подтверждено выводом утилиты `ifconfig` и содержимым файла `/var/lib/kea/kea-leases4.csv`.

Дополнительно настроено динамическое обновление DNS с использованием Kea DHCP-DDNS и TSIG-ключа `DHCP_UPDATER`. При получении клиентом DHCP-адреса в DNS-зоне `emcikunova.net` автоматически создаётся запись `client.emcikunova.net`, связывающая имя клиента с адресом `192.168.1.30`. Проверка командой `dig @192.168.1.1 client.emcikunova.net` подтверждает корректную работу динамического обновления DNS.

Действия по установке и настройке DHCP-сервера зафиксированы в `provisioning`-сценарии `dhcp.sh`, а настройки маршрутизации клиента — в сценарии `01-routing.sh`. Это позволяет автоматически воспроизводить конфигурацию DHCP-сервера и сетевого окружения при развёртывании виртуальных машин с помощью Vagrant.

4 Контрольные вопросы

1. В каких файлах хранятся настройки сетевых подключений?

В современных системах семейства RHEL, использующих NetworkManager, настройки сетевых подключений обычно хранятся в каталоге `/etc/NetworkManager/system-connections/`.

Для каждого подключения создаётся отдельный файл с расширением `.nmconnection`. Например, параметры интерфейса `eth1` могут находиться в файле:

```
/etc/NetworkManager/system-connections/eth1.nmconnection
```

В таких файлах содержатся имя и тип подключения, MAC-адрес, параметры IPv4 и IPv6, способ получения адреса, DNS-серверы, маршруты, шлюз и другие настройки.

Просматривать и изменять параметры подключений также можно с помощью утилиты `nmcli`. Например:

```
nmcli connection show
```

выводит список известных NetworkManager сетевых подключений, а команда

```
nmcli connection show eth1
```

показывает подробные параметры подключения `eth1`.

2. За что отвечает протокол DHCP?

DHCP (Dynamic Host Configuration Protocol) — протокол динамической конфигурации сетевых узлов. Он позволяет автоматически передавать клиентским устройствам параметры, необходимые для работы в IP-сети.

С помощью DHCP клиент может получить IP-адрес, маску сети, адрес шлюза по умолчанию, адрес DNS-сервера, доменное имя, домен поиска и другие сетевые параметры.

Использование DHCP позволяет не задавать сетевые настройки каждого клиентского узла вручную и уменьшает вероятность конфликтов IP-адресов. Сервер контролирует пул доступных адресов и предоставляет их клиентам на определённый срок, называемый временем аренды.

3. Поясните принцип работы протокола DHCP. Какими сообщениями обмениваются клиент и сервер, используя протокол DHCP?

При первоначальном подключении клиента к сети процесс получения адреса обычно состоит из четырёх основных этапов, которые часто обозначают как DORA: Discover, Offer, Request, Acknowledgement.

Сначала клиент, ещё не имеющий собственного IP-адреса, отправляет широковещательное сообщение DHCPDISCOVER. С его помощью клиент пытается обнаружить доступные DHCP-серверы в сети.

Получив запрос, DHCP-сервер выбирает свободный адрес из настроенного пула и отправляет клиенту сообщение DHCPOFFER. В нём сервер предлагает IP-адрес и дополнительные сетевые параметры, например маску сети, шлюз, DNS-сервер и срок аренды.

Клиент выбирает полученное предложение и отправляет сообщение DHCPREQUEST. Этим сообщением он сообщает, какой предложенный адрес хочет использовать.

После этого сервер подтверждает выделение адреса сообщением DHCPACK. Клиент применяет полученные параметры и начинает использовать выданный IP-адрес до окончания срока аренды.

Последовательность основного обмена имеет вид:

DHCPDISCOVER → DHCPOFFER → DHCPREQUEST → DHCPACK

Кроме основных сообщений существуют дополнительные сообщения DHCP. DHCPNAK используется сервером для отказа клиенту в запрошенной конфигу-

рации. DHCPDECLINE отправляется клиентом, если он обнаружил, что предложенный адрес уже используется другим узлом. DHCPRELEASE позволяет клиенту добровольно освободить полученный адрес. DHCPINFORM используется клиентом, который уже имеет IP-адрес, но хочет получить от DHCP-сервера дополнительные параметры конфигурации.

4. В каких файлах обычно находятся настройки DHCP-сервера? За что отвечает каждый из файлов?

В данной работе используется DHCP-сервер Kea. Его основные конфигурационные файлы располагаются в каталоге `/etc/kea/`.

Файл `/etc/kea/kea-dhcp4.conf` содержит основную конфигурацию DHCPv4-сервера. В нём задаются обслуживаемые сетевые интерфейсы, подсети, диапазоны выдаваемых IP-адресов, шлюзы, DNS-серверы, домены поиска, время аренды и параметры динамического обновления DNS.

Например, в выполненной работе в этом файле задана сеть `192.168.1.0/24` и пул:

`192.168.1.30 - 192.168.1.199`

Файл `/etc/kea/kea-dhcp-ddns.conf` содержит параметры службы Kea DHCP-DDNS. Она отвечает за передачу изменений о DHCP-клиентах DNS-серверу. В этом файле задаются прямая и обратная DNS-зоны, адрес DNS-сервера и используемый TSIG-ключ.

Файл `/etc/kea/tsig-keys.json` содержит описание TSIG-ключа, применяемого Kea для подписания запросов динамического обновления DNS.

Информация о фактически выданных IPv4-адресах хранится в файле:

`/var/lib/kea/kea-leases4.csv`

Это не основной конфигурационный файл, а база аренд. В ней содержатся выданный IP-адрес, MAC-адрес клиента, срок действия аренды, идентификатор подсети, имя клиента и другая служебная информация.

Для IPv6-сервера Kea используется отдельный конфигурационный файл `/etc/kea/kea-dhcp6.conf`.

5. Что такое DDNS? Для чего применяется DDNS?

DDNS (Dynamic DNS) — механизм динамического обновления записей DNS без необходимости вручную редактировать файлы DNS-зон при каждом изменении адреса узла.

DDNS особенно полезен совместно с DHCP. DHCP-сервер динамически выдаёт клиенту IP-адрес, после чего информация об имени и полученном адресе автоматически передаётся DNS-серверу. Благодаря этому DNS-записи соответствуют текущей конфигурации сети.

В выполненной работе клиент получает от Kea адрес 192.168.1.30. После получения аренды Kea DHCP-DDNS автоматически передаёт BIND запрос на обновление DNS-зоны. В результате создаётся запись:

```
client.emcikunova.net. → 192.168.1.30
```

Для защиты динамических обновлений используется TSIG-ключ DHCP_UPDATER. DNS-сервер принимает изменение зоны только при наличии корректной криптографической подписи.

DDNS применяется в сетях, где IP-адреса узлов могут изменяться и требуется автоматически поддерживать соответствие между их доменными именами и текущими адресами.

6. Какую информацию можно получить, используя утилиту ifconfig? Приведите примеры с использованием различных опций.

Утилита ifconfig предназначена для просмотра и изменения параметров сетевых интерфейсов.

Команда

```
ifconfig
```

показывает информацию об активных сетевых интерфейсах. Для каждого интерфейса можно увидеть его IPv4- и IPv6-адреса, маску сети, широковещательный адрес, MAC-адрес, значение MTU, состояние интерфейса и статистику передачи данных.

Например, в выполненной работе команда ifconfig на машине client пока-

зывает для интерфейса eth1:

```
inet 192.168.1.30
```

Это означает, что интерфейс получил от DHCP-сервера адрес 192.168.1.30.

Строка netmask показывает маску сети, а broadcast — широковещательный адрес. Поле ether содержит MAC-адрес сетевого интерфейса.

Поля RX packets и TX packets показывают количество принятых и переданных пакетов. Значения RX bytes и TX bytes содержат соответствующий объём данных. Поля errors, dropped, overruns и collisions позволяют обнаружить некоторые проблемы сетевого обмена.

Для вывода информации о конкретном интерфейсе используется команда:

```
ifconfig eth1
```

Она выводит параметры только интерфейса eth1.

Команда

```
ifconfig -a
```

показывает все известные сетевые интерфейсы, включая неактивные.

Для краткого представления статистики можно использовать:

```
ifconfig -s
```

В этом случае данные об интерфейсах выводятся в компактной табличной форме.

Утилита также позволяет изменять состояние интерфейса. Например:

```
ifconfig eth1 up
```

включает интерфейс eth1, а

```
ifconfig eth1 down
```

отключает его.

В современных Linux-системах для управления сетевыми интерфейсами вместо ifconfig часто применяются команды ip и nmcli, однако ifconfig остаётся удобной диагностической утилитой.

7. Какую информацию можно получить, используя утилиту ping? Приведите примеры с использованием различных опций.

Утилита `ping` используется для проверки сетевой доступности удалённого узла с помощью ICMP Echo Request и ICMP Echo Reply.

Например:

```
ping dhcr.emcikunova.net
```

позволяет проверить разрешение доменного имени и доступность соответствующего узла. Если DNS работает корректно, перед началом обмена отображается IP-адрес, в который было преобразовано доменное имя.

Для каждого полученного ответа `ping` показывает количество переданных байтов, адрес источника, порядковый номер `icmp_seq`, значение `ttl` и время прохождения пакета `time`.

Параметр `ttl` показывает оставшееся время жизни IP-пакета, выраженное в количестве допустимых переходов между маршрутизаторами. Параметр `time` показывает время между отправкой ICMP-запроса и получением ответа.

После завершения работы выводится общая статистика: количество отправленных и принятых пакетов, процент потерь и статистика времени прохождения пакетов.

Чтобы ограничить количество запросов, используется параметр `-c`. Например:

```
ping -c 4 192.168.1.1
```

отправляет четыре ICMP-запроса и после этого автоматически завершает работу.

Параметр `-i` позволяет изменить интервал между запросами:

```
ping -i 2 192.168.1.1
```

В этом случае запросы отправляются с интервалом две секунды.

Для принудительного использования IPv4 можно выполнить:

```
ping -4 client.emcikunova.net
```

Для IPv6 используется:

```
ping -6 <IPv6-адрес>
```

Параметр `-s` позволяет изменить размер данных ICMP-пакета. Например:

```
ping -s 1000 192.168.1.1
```

отправляет запросы с полезной нагрузкой размером 1000 байт.

Параметр -W задаёт время ожидания ответа на отдельный запрос:

```
ping -W 2 192.168.1.1
```

В данном примере ответ ожидается в течение двух секунд.

Следовательно, с помощью ping можно проверить доступность узла, работоспособность DNS-разрешения при использовании доменного имени, оценить задержку и потери пакетов, а также косвенно диагностировать проблемы сетевого соединения.